INTEGRATING RABBITMQ WITH SPRING

First, open the build.gradle file for the Dispatcher Service project (dispatcher-service), and replace the Spring Cloud Function dependency with the RabbitMQ binder for Spring Cloud Stream. Since Spring Cloud Function is already included in Spring Cloud Stream, you don’t need to add it explicitly. You can also remove the dependency on Spring Boot Starter, which is also included in the Spring Cloud Stream dependency. Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.12 Updating dependencies in Dispatcher Service

```
dependencies {
  implementation
  ➥ 'org.springframework.cloud:spring-cloud-stream-binder-rabbit'
  testImplementation 'org.springframework.boot:spring-boot-starter-test'
  testImplementation 'io.projectreactor:reactor-test'
}
```

Next, open the application.yml file and add the following configuration for the RabbitMQ integration. Port, username, and password are the same ones we previously defined in Docker Compose (listings 10.1 and 10.2).

Listing 10.13 Configuring the RabbitMQ integration

```
spring:
  rabbitmq:
    host: localhost
    port: 5672
    username: user
    password: password
    connection-timeout: 5s
```

That’s it. If you run the Dispatcher Service, you’ll notice it already works perfectly without further configuration. Spring Cloud Stream will auto-generate and configure the bindings to exchanges and queues in RabbitMQ.